ECSE 222, Digital Logic

VHDL 3:

Adders and Critical Path

Fall 2023
University of McGill

Presented by:

Nara Yun (Student #: 261115268)

Karen Chen Lai (Student #: 261107674)

Instructors: Prof. Boris Vaisband

Teaching Assistant: Maninder Bir Singh Gulshan

Date: October 24rd, 2023

Executive Summary

This experiment consists of a few parts with a final purpose of advancing our practice and understanding of digital circuit design by implementing and simulating complex adder circuits. In this assignment, we implemented half adder, full adder, 4-bit Ripple-Carry Adder, one-digit BCD Adder and critical path of digital circuits.

First part consists of the design of a 4-bit Ripple-Carry Adder (RCA). We implemented a structural description of a 4-bit RCA, utilizing basic building blocks like half-adders and full-adders. Then we designed and tested a VHDL structural model for the 4-bit RCA, incorporating thorough testing to ensure functionality. Moreover, we crafted a behavioral VHDL description of the same 4-bit RCA circuit, this time using arithmetic operators. A comprehensive testbench was created for the behavioral description, conducting exhaustive tests to validate its performance.

The second part is the design of a One-Digit Binary Coded Decimal (BCD) Adder. We constructed a structural description for the BCD adder, offering the flexibility to use either structural or behavioral coding styles. A detailed VHDL code was implemented for the structural BCD adder, and extensive testing was performed to validate its functionality. In addition, we devised a behavioral VHDL description for the BCD adder circuit, with a focus on conditional signal assignments. Same as the first part, we constructed a testbench for conducting exhaustive tests for BCD adder descriptions.

The last part is the critical Path Analysis. We learned how to utilize Quartus Timing Analyzer tools to identify the critical path in digital circuits. Timing constraints were introduced to gauge latency, and slack values were determined to evaluate if timing requirements were met. We examined and reported the critical path of the one-digit BCD adder circuit and visually presented the timing waveforms associated with it.

Questions

1. Briefly explain your VHDL code implementation of all circuits.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_half_adder is
    port(a: in std_logic;
         b: in std_logic;
         s: out std_logic;
         c: out std_logic);
end Karen_Chen_half_adder;

architecture structural of Karen_Chen_half_adder is
begin
      process (a,b)
      begin
         s <= a xor b;
         c <= (a and b);
      end process;
end;
```

Figure 1. Structural description of half adder

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_half_adder_tb is
--empty
   end Karen_Chen_half_adder_tb;

architecture tb of Karen_Chen_half_adder_tb is

   component Karen_Chen_half_adder
      port( a, b: in std_logic;
            s, c: out std_logic);
   end component;

   signal a_in, b_in, s_out, c_out : std_logic;


   begin
   dut: Karen_Chen_half_adder port map(
      a => a_in,
      b => b_in,
      c => c_out,
      s => s_out);

      test: process
      begin
         for i in std_logic range '0'to '1' loop
            a_in <= i;
            for j in std_logic range '0' to '1' loop
               b_in <= j;
               wait for 10 ns;
            end loop;
         end loop;
      wait;
   end process;

end tb;
```

Figure 2. Testbench for structural description of half adder

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_full_adder is
    port(a: in std_logic;
         b: in std_logic;
         c_in: in std_logic;
         s: out std_logic;
         c_out: out std_logic);
    end Karen_Chen_full_adder;

architecture structural of Karen_Chen_full_adder is

    component Karen_Chen_half_adder
        port(a, b : in std_logic;
             s, c: out std_logic);
    end component;

    signal sum, c_HA1, c_HA2: std_logic;

begin

        HA1: Karen_Chen_half_adder
            port map(a, b, sum, c_HA1);


        HA2: Karen_Chen_half_adder
            port map(sum, c_in, s, c_HA2);

        c_out <= c_HA1 or c_HA2;

end;
```

Figure 3. Structural description of full adder

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_full_adder_tb is
--empty
    end Karen_Chen_full_adder_tb;

architecture tb of Karen_Chen_full_adder_tb is

    component Karen_Chen_full_adder
        port( a: in std_logic;
            b: in std_logic;
            c_in: in std_logic;
            s: out std_logic;
            c_out: out std_logic);
    end component;

    signal a_in, b_in, c_in_in, s_out, c_out_out : std_logic;


    begin
    dut: Karen_Chen_full_adder port map(
        a => a_in,
        b => b_in,
        c_in => c_in_in,
        c_out => c_out_out,
        s => s_out);

        test: process
        begin
            for i in std_logic range '0'to '1' loop
                a_in <= i;
                for j in std_logic range '0' to '1' loop
                    b_in <= j;
                    for k in std_logic range '0' to '1' loop
                        c_in_in <= k;
                        wait for 10 ns;
                    end loop;
                end loop;
            end loop;
        wait;
    end process;

end tb;
```

Figure 4. Testbench for structural description of full adder

Figure 1 and 3 illustrate the structural descriptions for half adder and full adder respectively. The half adder takes two inputs and produces one sum and one carry out, in which the sum is generated by the XOR gate and carry out is produced by an AND gate. Whereas the full adder takes 2 inputs, 1 carry-in and produces 1 carry out and 1 sum. It implements 2 half adders and 1 OR gate. Figure 3 and 4 are the testbench codes that perform exhaustive tests for the structural descriptions.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_rca_structural is
    port(
        A, B: in std_logic_vector (3 downto 0);
        S: out std_logic_vector (4 downto 0)
    );
end Karen_Chen_rca_structural;

architecture rca_struct of Karen_Chen_rca_structural is
    -- use full adders in RCA
    component Karen_Chen_full_adder
        port (
            a,b, c_in: in std_logic;
            s,c_out: out std_logic);
    end component;

    -- carry/temporary variables
    signal c_FA1, c_FA2, c_FA3: std_logic;

begin
    -- order of mapping: a, b, c_in, s, c_out
    -- pattern of mapping:
    -- FA_i --> A(i), B(i), c_FA(i-1), S(i), c_FA(i)

    -- Could have used a half_adder ;)
    FA1: Karen_Chen_full_adder
        port map(A(0), B(0), '0', S(0), c_FA1);
         -- c_in of FA1 is '0'

    FA2: Karen_Chen_full_adder
        port map(A(1), B(1), c_FA1, S(1), c_FA2);

    FA3: Karen_Chen_full_adder
        port map(A(2), B(2), c_FA2, S(2), c_FA3);

    FA4: Karen_Chen_full_adder
        port map(A(3), B(3), c_FA3, S(3), S(4));
end;
```

Figure 5. Structural description of 4-bit Ripple-Carry-Adder

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_rca_structural_tb is
--empty
end Karen_Chen_rca_structural_tb;

-- Testbench for the half-adder
architecture tb of Karen_Chen_rca_structural_tb is
    component Karen_Chen_rca_structural
        port(
            A, B: in std_logic_vector (3 downto 0);
            S:    out std_logic_vector (4 downto 0)
        );
    end component;

    SIGNAL A_in, B_in:   std_logic_vector (3 downto 0);
    SIGNAL S_out:        std_logic_vector (4 downto 0);

begin
    -- device under test
    DUT: Karen_Chen_rca_structural
        port map(A_in, B_in, S_out);

    Simuli: process
    begin
        report "STRUCTURAL RCA using 4 Full-Adders test";
        for i in 0 to 15 loop --2^4(bit width)-1 = 15
            A_in <= std_logic_vector(to_unsigned(i,4));
            for j in 0 to 15 loop
                B_in <= std_logic_vector(to_unsigned(j,4));
                wait for 10 ns;
            end loop;
        end loop;
        report "FINISHED!";
        wait;
    end process;
end;
```

Figure 6. Testbench for structural description of 4-bit RCA

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

--use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity Karen_Chen_rca_behavioral is
    port(
        A, B: in std_logic_vector (3 downto 0);
        S:    out std_logic_vector (4 downto 0)
    );
end Karen_Chen_rca_behavioral;

architecture rca_behave of Karen_Chen_rca_behavioral is

    --signal S_temp: std_logic_vector (4 downto 0);
begin
    S<= '0' & A + B;
end;
```

Figure 7. Behavioral description of 4-bit Ripple-Carry-Adder

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_rca_behavioral_tb is
--empty
end Karen_Chen_rca_behavioral_tb;

-- Testbench for the half-adder
architecture tb of Karen_Chen_rca_behavioral_tb is
    component Karen_Chen_rca_behavioral
        port(
            A, B: in std_logic_vector (3 downto 0);
            S:    out std_logic_vector (4 downto 0)
        );
    end component;

    SIGNAL A_in, B_in:   std_logic_vector (3 downto 0);
    SIGNAL S_out:        std_logic_vector (4 downto 0);

begin
    -- device under test
    DUT: Karen_Chen_rca_behavioral
        port map(A_in, B_in, S_out);

    Simuli: process
    begin
        report "BEHAVIORAL RCA test";
        for i in 0 to 15 loop --2^4(bit width)-1 = 15
            A_in <= std_logic_vector(to_unsigned(i,4));
            for j in 0 to 15 loop
                B_in <= std_logic_vector(to_unsigned(j,4));
                wait for 10 ns;
            end loop;
        end loop;
        report "FINISHED!";
        wait;
    end process;
end;
```

Figure 8. Testbench for behavioral description of 4-bit RCA

Figure 5 is the structural description of 4-bit RCA in which it implements 4 full adders and describes the necessary signals that connect the ports. Figure 7 shows the implementation through operators such that the sum produced by the RCA has 5 bits by adding a '0' at the left-hand side of input A and then operate addition with the input B. Figure 6 and 8 illustrates the testbench code for the 4-bit Ripple Carry Adder.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_bcd_adder_structural is
    port(
        A, B: in std_logic_vector (3 downto 0);
        S: out std_logic_vector (3 downto 0);
        C: out std_logic
    );
end Karen_Chen_bcd_adder_structural;

architecture bcd_struct of Karen_Chen_bcd_adder_structural is
    component Karen_Chen_rca_structural
        port(
        A, B: in std_logic_vector (3 downto 0);
        S: out std_logic_vector (4 downto 0)
    );
    end component;

    -- Using a Half_adder and a Full_adder to implement a 2-bit adder
    component Karen_Chen_Half_Adder
        port(
            a,b: in std_logic;
            s,c: out std_logic);
    end component;

    component Karen_Chen_Full_Adder
        port(
            a,b, c_in: in std_logic;
            s,c_out: out std_logic);
    end component;

    signal Z:   std_logic_vector(4 downto 0);
    signal HA_c, FA_c, C_temp: std_logic;

begin
    -- mapping patterns:
    -- A of BCD => A of RCA
    -- B of BCD => B of RCA
    -- Z => S of RCA
    -- (A, B, Z)
    RCA1: Karen_Chen_rca_structural
        port map(A, B, Z);

    -- Detect if sum is > 9, yields MUX_input
    -- Z(4) (the carry out) = true: s_temp > 15
    -- 2 other conditions: 9 < s_temp < 16
    C_temp <=   Z(4) OR (Z(3) and (Z(2) or Z(1)));

    -- Two-bit adder, yields temporary variable S(1) and S(2)
    HA: Karen_Chen_Half_Adder -- map(a,b,s,c)
        port map(C_temp, Z(1), S(1), HA_c);

    FA: Karen_Chen_Full_Adder -- map(a, b, c_in, s, c_out)
        port map(C_temp, Z(2), HA_c, S(2), FA_c);

    -- Getting S(3)
    S(3) <= Z(3) xor FA_c;

    -- Mapping the output
    S(0) <= Z(0);
    C <= C_temp;
end;
```

Figure 9. Structural description of a BCD Adder

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_bcd_adder_structural_tb is
-- empty
end Karen_Chen_bcd_adder_structural_tb;

architecture tb of Karen_Chen_bcd_adder_structural_tb is
    component Karen_Chen_bcd_adder_structural
        port(
            A, B: in std_logic_vector (3 downto 0);
            S:    out std_logic_vector (3 downto 0);
            C:    out std_logic);
    end component;

    -- Testing variables
    signal A_in, B_in, S_out:  std_logic_vector(3 downto 0);
    signal C_out:              std_logic;

begin
    dut: Karen_Chen_bcd_adder_structural
        port map(A_in, B_in, S_out, C_out);

-- This is how the test should proceed:
-- (+) The BCD adder only adds numbers from 0 to 9
-- (+) If the input is greater than 9, the output should be don't care '-'
-- (+) Exhaustive test produces all possible inputs as well as all possible output

    simulate: process
    begin
        report "STRUCTURAL One-digit BCD adder Exhaustive Test";
        for i in 0 to 9 loop
            A_in <= std_logic_vector(to_unsigned(i,4));
            for j in 0 to 9 loop
                B_in <= std_logic_vector(to_unsigned(j,4));
                wait for 10 ns;
            end loop;
        end loop;
        wait;
    end process;
end;
```

Figure 10. Testbench for structural description of BCD Adder

Figure 9 demonstrates the structural description of a BCD Adder. In which it takes two 4-bit numbers into a 4-bit RCA component to produce intermediate results. Then operate separately with half adder and full adder in order to produce the final results: S_0 is same as intermediate result, Z_0; S_1 and S_2 are produced by a 2-bit RCA; S_3 is yield by a XOR gate with the intermediate Z_3 and carry out of the 2-bit CRA; finally the final carry out is produced by Z_4 OR (Z_3 AND (Z_2 OR Z_1))) as shown in line 49. Figure 10 illustrates the testbench for structural description of BCD adder.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4  use ieee.std_logic_unsigned.all;
5
6  entity Karen_Chen_bcd_adder_behavioral is
7     port(
8        A, B: in std_logic_vector (3 downto 0);
9        S: out std_logic_vector (3 downto 0);
10       C: out std_logic
11    );
12 end Karen_Chen_bcd_adder_behavioral;
13
14 architecture bcd_behave of Karen_Chen_bcd_adder_behavioral is
15    signal temp: std_logic_vector(4 downto 0);
16    signal dont_care: std_logic_vector(3 downto 0) := "----"; --dont care--
17
18 begin
19
20    process(A,B,temp)
21
22    begin
23       if (A > "1001" OR B > "1001") then
24          S <= dont_care;
25          C <= '-';
26       else
27          temp <= ('0' & A) + B;
28          if temp < "01010" then
29             C <= '0';
30             S <= A + B;
31          elsif temp > "01001" then
32             C <= '1';
33             S <= A + B + "0110";
34          end if;
35       end if;
36
37    end process;
38 end;
```

Figure 11. Behavioral description of BCD Adder

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity Karen_Chen_bcd_adder_behavioral_tb is
6  --empty
7  end Karen_Chen_bcd_adder_behavioral_tb;
8
9  architecture tb of Karen_Chen_bcd_adder_behavioral_tb is
10    component Karen_Chen_bcd_adder_behavioral
11       port(
12          A, B: in std_logic_vector (3 downto 0);
13          S:    out std_logic_vector (3 downto 0);
14          C:    out std_logic);
15    end component;
16
17    -- Testing variables
18    signal A_in, B_in, S_out:  std_logic_vector(3 downto 0);
19    signal C_out:              std_logic;
20
21 begin
22    dut: Karen_Chen_bcd_adder_behavioral
23       port map(A_in, B_in, S_out, C_out);
24
25
26    simulate: process
27    begin
28       report "BEHAVIORAL One-digit BCD adder Exhaustive Test";
29       for i in 0 to 15 loop
30          A_in <= std_logic_vector(to_unsigned(i,4));
31          for j in 0 to 15 loop
32             B_in <= std_logic_vector(to_unsigned(j,4));
33             wait for 10 ns;
34          end loop;
35       end loop;
36       wait;
37    end process;
38 end;
```

Figure 12. Testbench for behavioral description of BCD Adder

Figure 11 demonstrates the behavioral architecture of BCD Adder, in which check whether the addition requires to add 00110, 6 in decimal, in order to convert binary number to BCD if it is greater than 9. Figure 12 is the testench code that perform exhaustive tests for validating BCD adder.

2. Show representative simulation plots of the half-adder circuit for all possible input values.

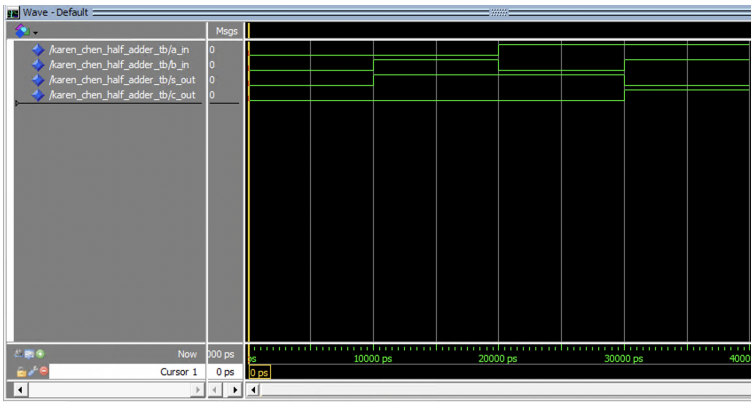


Figure 13. Simulation of exhaustive tests of half adder

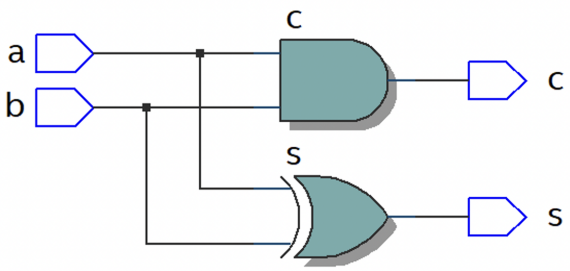


Figure 14. Schematic view of half adder

3. Show representative simulation plots of the full-adder circuit for all possible input values.

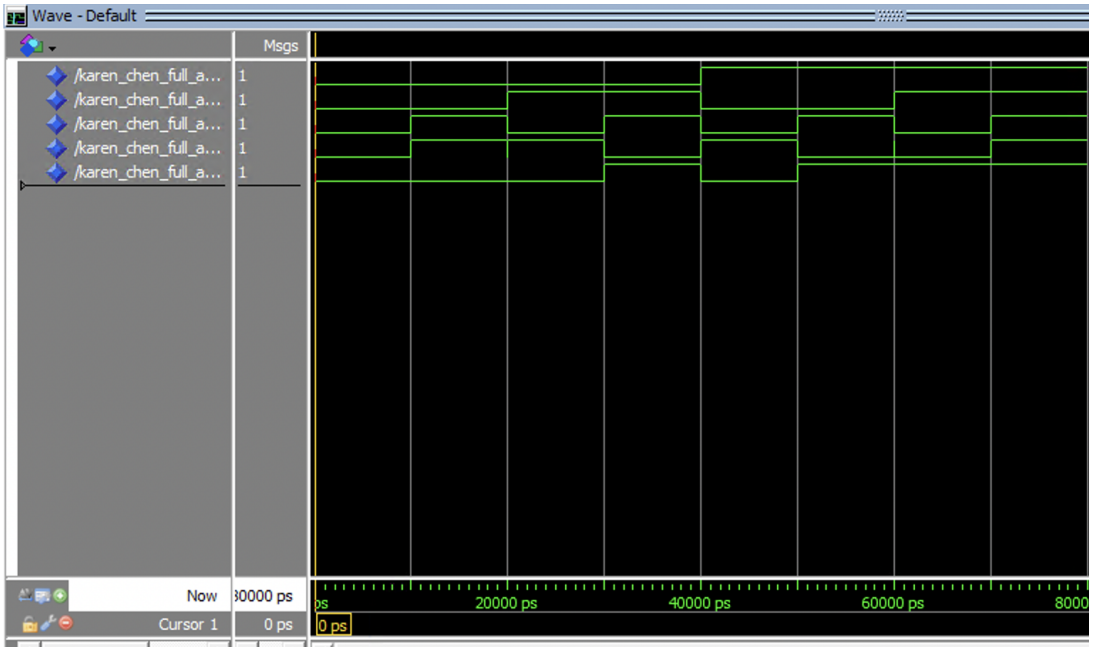


Figure 15. Simulation of exhaustive tests of full adder

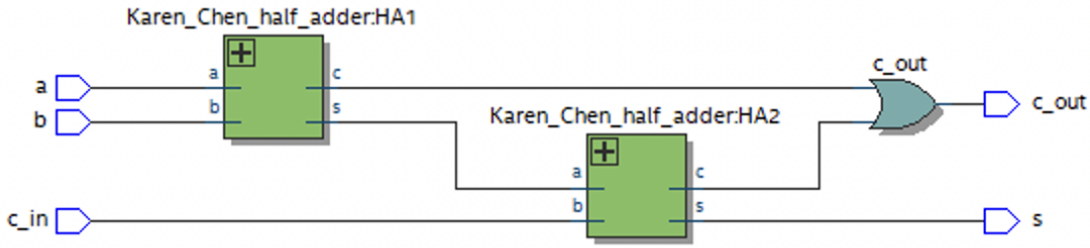


Figure 16. Schematic view of full adder

4. Show representative simulation plots of both behavioral and structural descriptions of the 4-bit RCA for all possible input values.

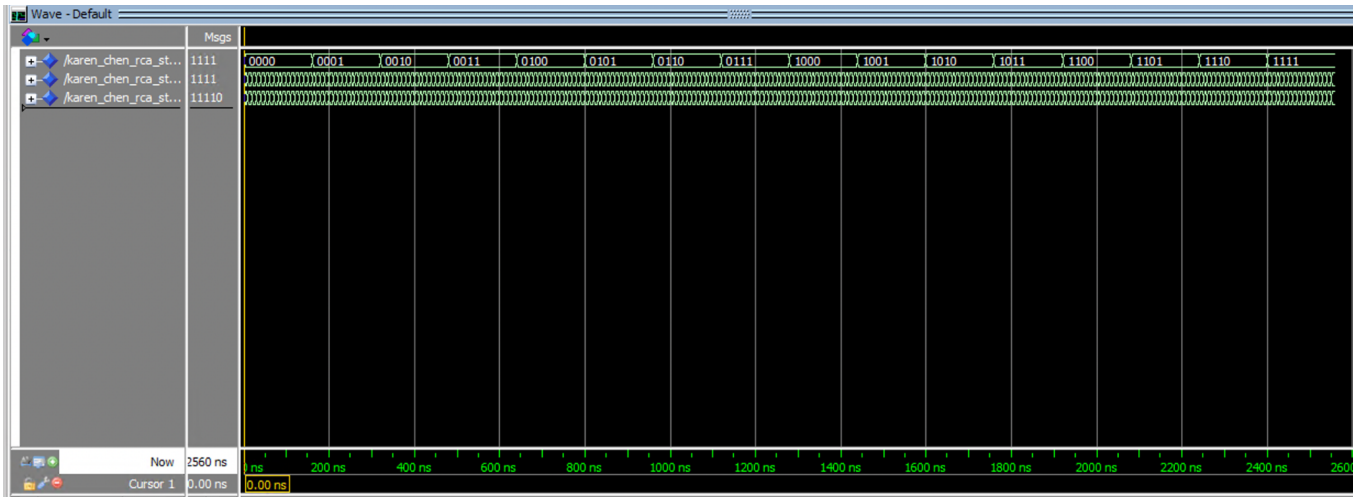


Figure 17. Simulation of exhaustive tests of structural description of RCA

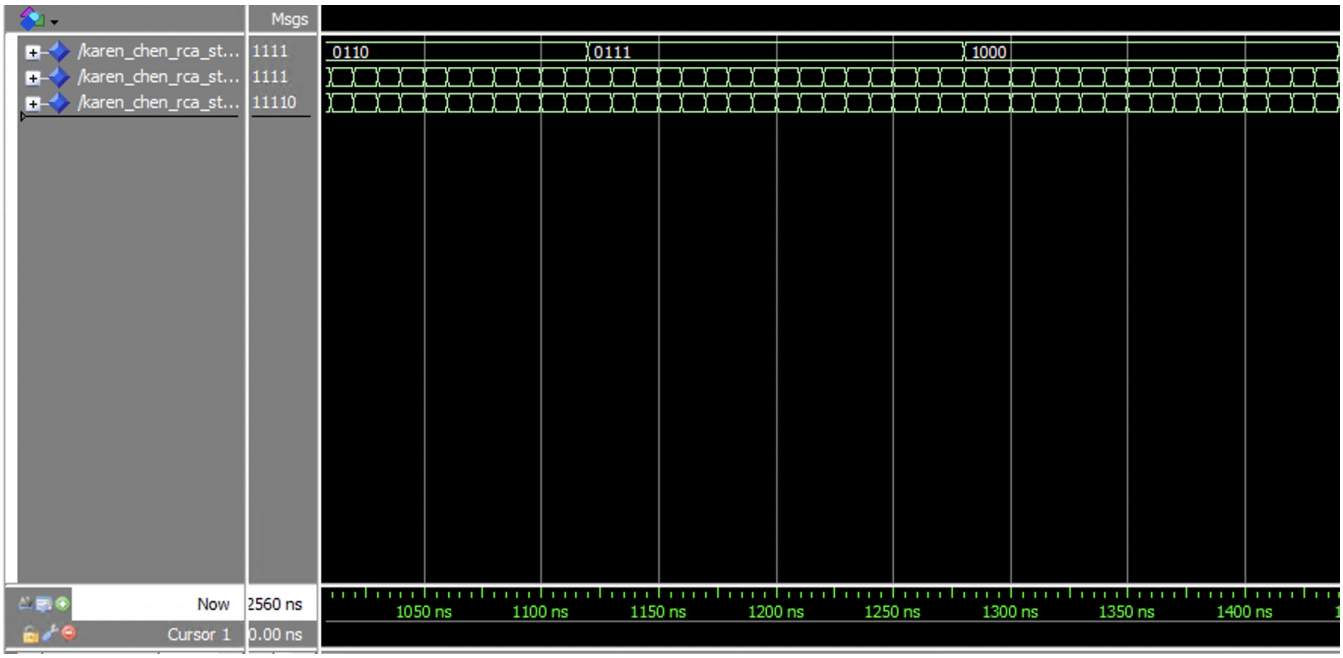


Figure 18. Simulation of exhaustive tests of RCA (Zoom-in ver.)

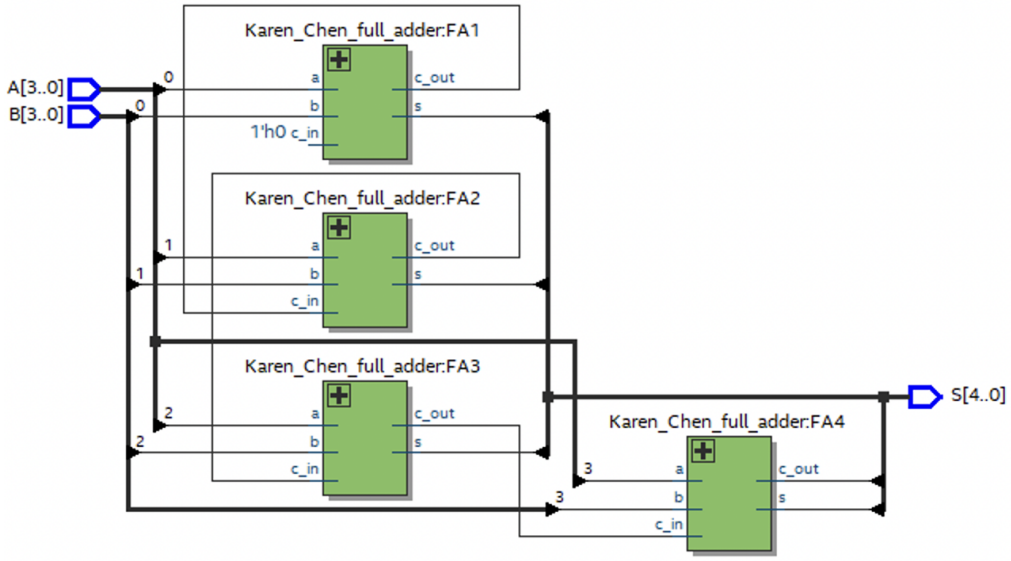


Figure 19. Schematic view of structural description of RCA

Figure 20. Simulation of exhaustive tests of behavioral description of RCA

Figure 21. Simulation of exhaustive tests of behavioral description of RCA (Zoom-in ver.)

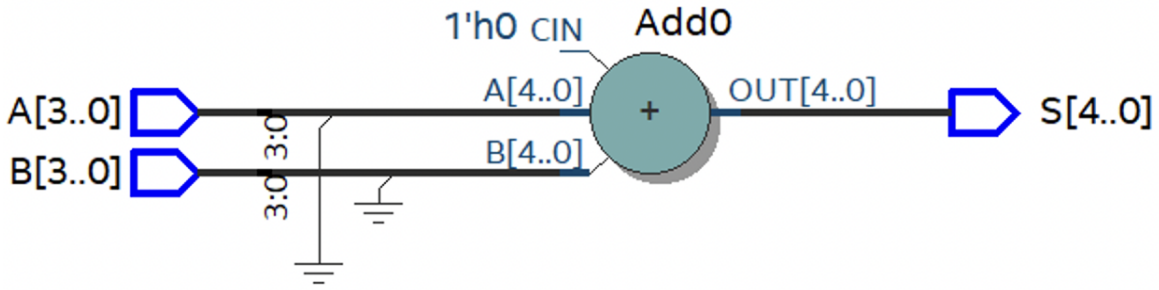


Figure 22. Schematic view of behavioral description of RCA

5. Show representative simulation plots of both behavioral and structural descriptions of the one-digit BCD adder circuit for all possible input values.

0Figure 23. Simulation of exhaustive tests of structural description of one-digit BCD

Figure 24. Simulation of exhaustive tests of structural description of one-digit BCD (Zoom-in ver.)

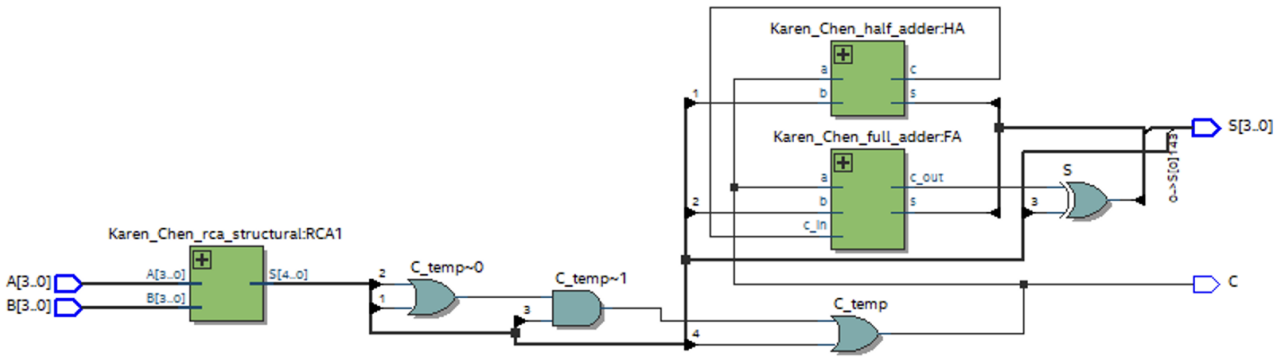


Figure 25. Schematic view of structural description of one-digit BCD

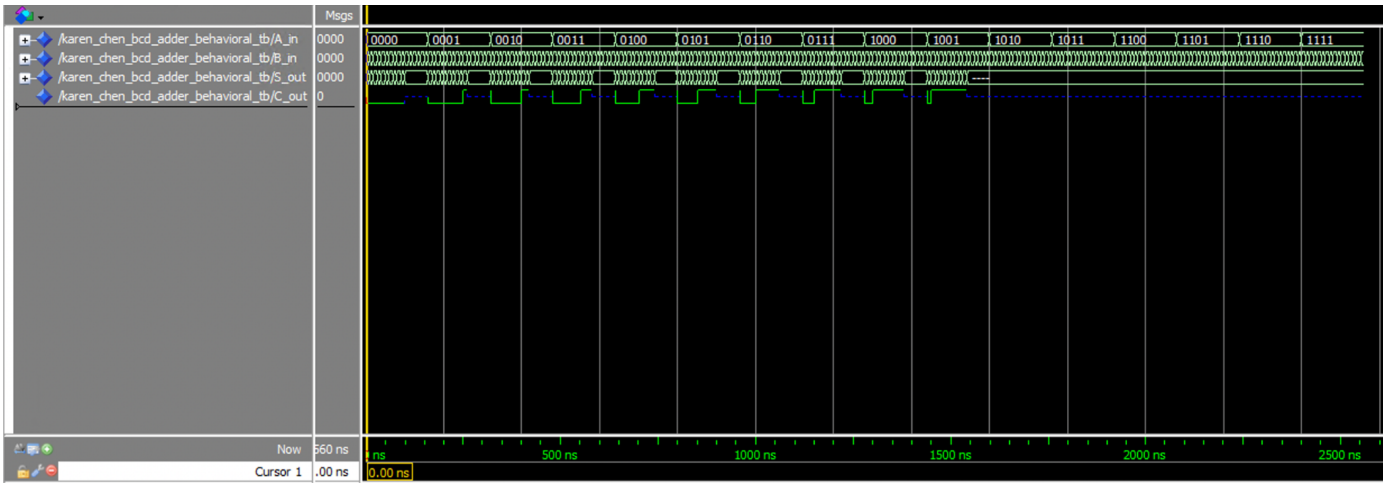


Figure 26. Simulation of exhaustive tests of behavioral description of one-digit BCD

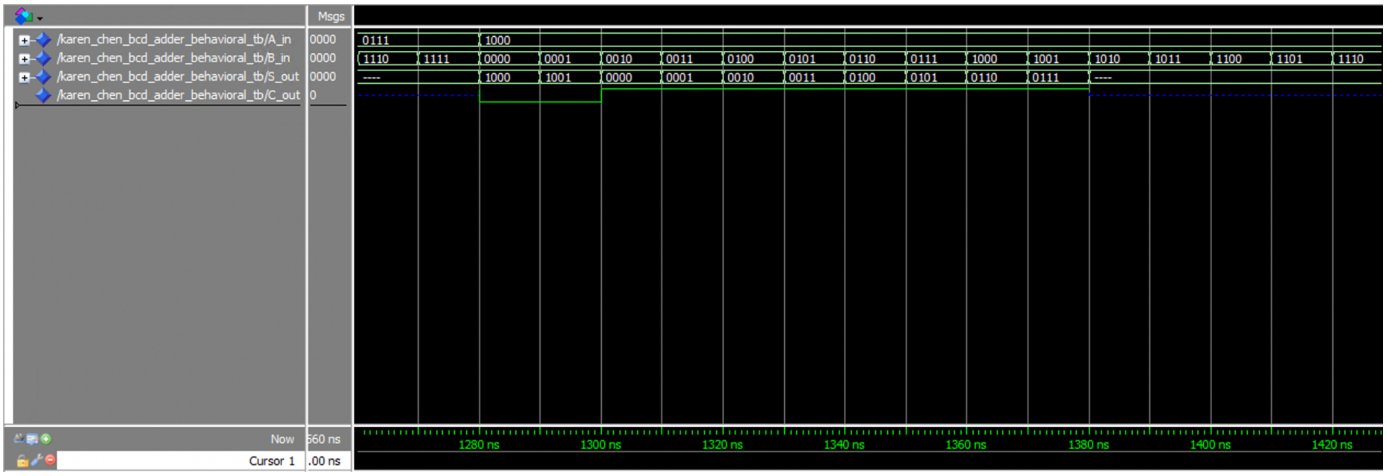


Figure 27. Simulation of exhaustive tests of behavioral description of one-digit BCD (Zoom-in ver.)

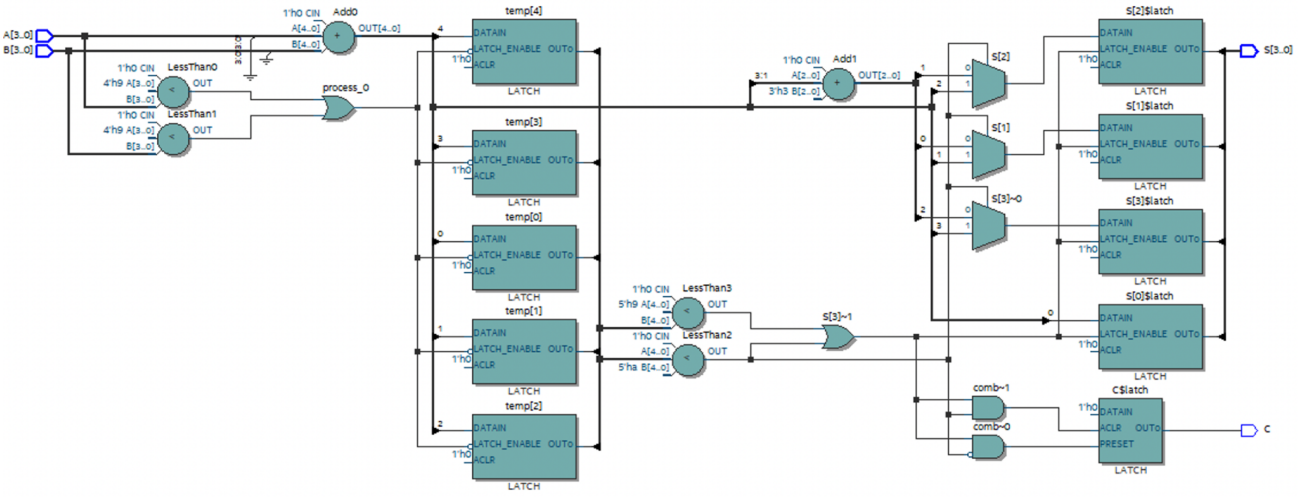


Figure 28. Schematic view of behavioral description of one-digit BCD

6. Perform timing analysis and find the critical path(s) of the one-digit BCD adder circuit for the Fast 1,100 mV 85C Model. Show the obtained timing waveform(s) of the critical path(s) that you found.

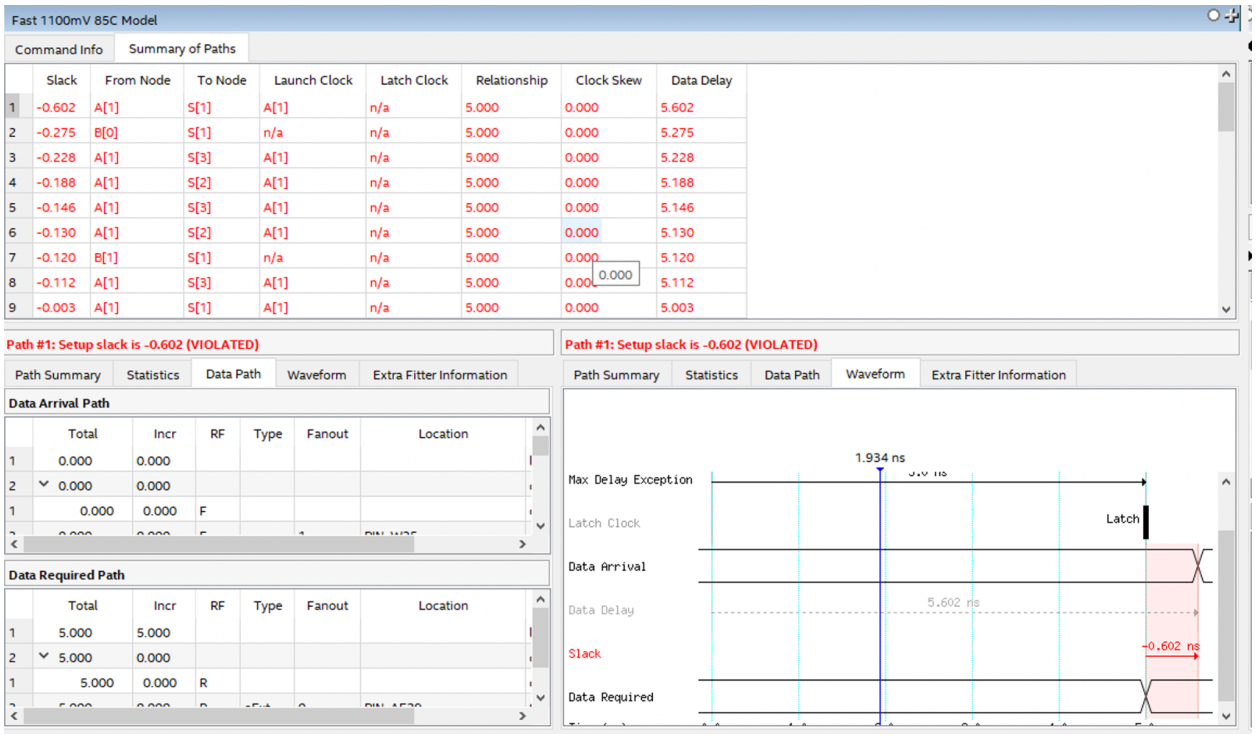
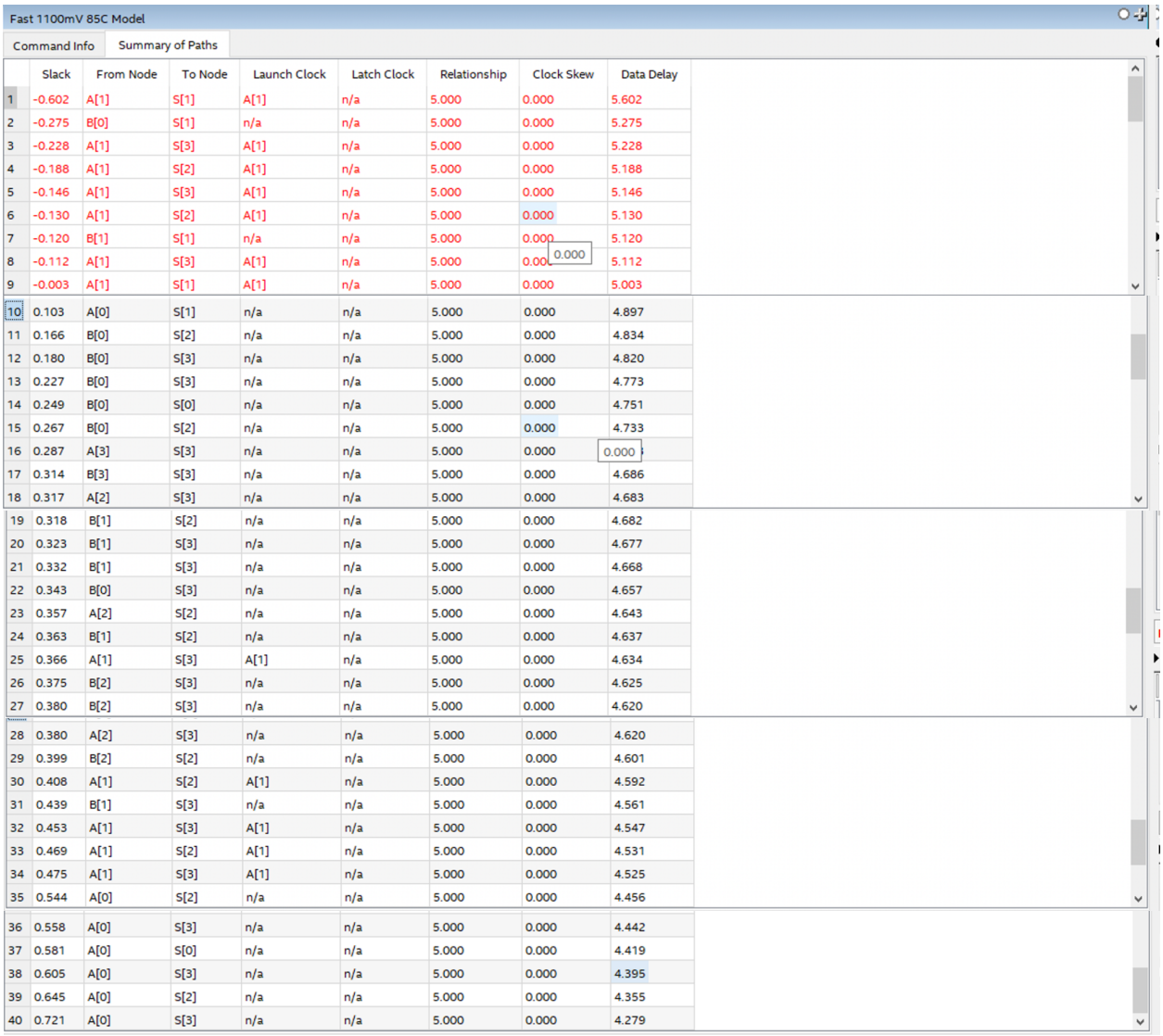


Figure 29. Timing waveform of the critical path in one-digit BCD


Fast 1100mV 85C Model

Command Info | Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-0.602	A[1]	S[1]	A[1]	n/a	5.000	0.000	5.602
2	-0.275	B[0]	S[1]	n/a	n/a	5.000	0.000	5.275
3	-0.228	A[1]	S[3]	A[1]	n/a	5.000	0.000	5.228
4	-0.188	A[1]	S[2]	A[1]	n/a	5.000	0.000	5.188
5	-0.146	A[1]	S[3]	A[1]	n/a	5.000	0.000	5.146
6	-0.130	A[1]	S[2]	A[1]	n/a	5.000	0.000	5.130
7	-0.120	B[1]	S[1]	n/a	n/a	5.000	0.000	5.120
8	-0.112	A[1]	S[3]	A[1]	n/a	5.000	0.000	5.112
9	-0.003	A[1]	S[1]	A[1]	n/a	5.000	0.000	5.003
10	0.103	A[0]	S[1]	n/a	n/a	5.000	0.000	4.897
11	0.166	B[0]	S[2]	n/a	n/a	5.000	0.000	4.834
12	0.180	B[0]	S[3]	n/a	n/a	5.000	0.000	4.820
13	0.227	B[0]	S[3]	n/a	n/a	5.000	0.000	4.773
14	0.249	B[0]	S[0]	n/a	n/a	5.000	0.000	4.751
15	0.267	B[0]	S[2]	n/a	n/a	5.000	0.000	4.733
16	0.287	A[3]	S[3]	n/a	n/a	5.000	0.000	4.713
17	0.314	B[3]	S[3]	n/a	n/a	5.000	0.000	4.686
18	0.317	A[2]	S[3]	n/a	n/a	5.000	0.000	4.683
19	0.318	B[1]	S[2]	n/a	n/a	5.000	0.000	4.682
20	0.323	B[1]	S[3]	n/a	n/a	5.000	0.000	4.677
21	0.332	B[1]	S[3]	n/a	n/a	5.000	0.000	4.668
22	0.343	B[0]	S[3]	n/a	n/a	5.000	0.000	4.657
23	0.357	A[2]	S[2]	n/a	n/a	5.000	0.000	4.643
24	0.363	B[1]	S[2]	n/a	n/a	5.000	0.000	4.637
25	0.366	A[1]	S[3]	A[1]	n/a	5.000	0.000	4.634
26	0.375	B[2]	S[3]	n/a	n/a	5.000	0.000	4.625
27	0.380	B[2]	S[3]	n/a	n/a	5.000	0.000	4.620
28	0.380	A[2]	S[3]	n/a	n/a	5.000	0.000	4.620
29	0.399	B[2]	S[2]	n/a	n/a	5.000	0.000	4.601
30	0.408	A[1]	S[2]	A[1]	n/a	5.000	0.000	4.592
31	0.439	B[1]	S[3]	n/a	n/a	5.000	0.000	4.561
32	0.453	A[1]	S[3]	A[1]	n/a	5.000	0.000	4.547
33	0.469	A[1]	S[2]	A[1]	n/a	5.000	0.000	4.531
34	0.475	A[1]	S[3]	A[1]	n/a	5.000	0.000	4.525
35	0.544	A[0]	S[2]	n/a	n/a	5.000	0.000	4.456
36	0.558	A[0]	S[3]	n/a	n/a	5.000	0.000	4.442
37	0.581	A[0]	S[0]	n/a	n/a	5.000	0.000	4.419
38	0.605	A[0]	S[3]	n/a	n/a	5.000	0.000	4.395
39	0.645	A[0]	S[2]	n/a	n/a	5.000	0.000	4.355
40	0.721	A[0]	S[3]	n/a	n/a	5.000	0.000	4.279

Figure 30. Summary of paths of the critical path in one-digit BCD

7. Report the number of pins and logic modules used to fit your designs on the FPGA board.

	RCA		One-digit BCD adder	
	Structural	**Behavioral**	**Structural**	**Behavioral**
Logic Utilization(in LUTs)	4/32070 (<1%)	3/32070 (<1%)	7/32070 (<1%)	9/32070 (<1%)
Total pins	13/457 (3%)	13/457 (3%)	13/457 (3%)	13/457 (3%)

Explanations of the Results

In Figure 13, the simulation plots for the Half-Adder circuit offer a comprehensive insight into its fundamental operation. They encompass a range of input combinations, including both '0' and '1' states for inputs A and B, which are essential to demonstrate the circuit's binary addition functionality. The Sum (S) output should correctly reflect the result of adding A and B, showing '1' when A and B are different and '0' when they are the same. The Carry (C) output should indicate the generation of a carry only when both A and B are '1.' The plots also reveal transition times and propagation delays, which are critical for assessing the circuit's speed. Consistency across all simulations is essential to confirm that the circuit operates as expected for all possible input combinations, aligning with the logical rules of binary addition. Additionally, when comparing simulation plots of both structural and behavioral implementations, any discrepancies or differences would highlight design issues, emphasizing the importance of correctness and equivalency in the designs.

In the Figure 15 which is simulation plots for a full-adder circuit, the waveforms of the input signals (A, B, and Cin) should accurately represent various combinations of binary inputs transitioning between logic levels '0' and '1.' These plots should demonstrate the full-adder's ability to add three inputs, including an optional carry-in (Cin). The output signals, Sum (S) and Carry-out (Cout), should exhibit their expected behavior. Specifically, when all inputs are '0,' both Sum and Cout should be '0,' indicating no carry and a sum of '0.' When appropriate inputs are '1,' the Sum should reflect the binary sum, while Cout should indicate the presence or absence of carry. For example, When A is ‘1’, B is ‘0 and Cin is ‘0’, the Sum should be ‘1’, and Cout shouldBe ‘0’, as there is no carryThe simulation plots should comprehensively test different input scenarios, ensuring that the full-adder adheres to binary addition rules and correctly handles carry propagation and generation.

In Figure 17 which is the simulation plots for the 4-bit Ripple-Carry Adder (RCA), both the structural and behavioral implementations should exhibit the expected behavior of an adder circuit. The waveforms of the input signals (A and B) should reflect various combinations of 4-bit binary inputs transitioning between logic levels '0' and '1.' These plots should effectively demonstrate the functionality of the 4-bit RCA, showcasing its ability to perform binary addition on multi-bit inputs. The Sum (S) output should represent the accurate sum of the inputs, adhering to binary addition rules. The Carry-out (Cout) signal should show the generation of carry when required. By comparing the results of the structural and behavioral implementations, the simulation plots should illustrate that both approaches yield the same correct output, emphasizing the functional equivalence of the two designs. This comprehensive testing ensures that the 4-bit RCA correctly handles carry propagation and generation across multiple bits, a fundamental aspect of arithmetic circuits.

In Figure 26, which is the simulation plots for the One-Digit BCD Adder (Structural and Behavioral) provide valuable insights into the functionality of this circuit. In the context of structural implementation, these plots illustrate the interconnection of basic logic components, demonstrating how the BCD Adder processes input data and produces output results. The simulation plots should confirm that the structural implementation adheres to the logical

principles of binary-coded decimal addition. They should show the formation of the correct BCD digits for each sum and the generation of a carry when the sum exceeds '9' in BCD format.

Conclusions

Through this experiment, we learned how to:

- Use Quartus Timing Analyzer tools to find the critical path.
- Create VHDL descriptions and testbenches for a ripple-carry in behavioral and structural styles through practice with Half Adder and Full Adder.
- Create VHDL descriptions and testbenches for BCD adders including behavioral and structural styles.
- Enhance our observations and comprehensions on the architecture sections through the debugging process and RTL simulation analysis.